ThreadLearn: A RAG-Augmented Fine-Tuned Language Model for JavaScript Concurrency Bug Detection and Remediation

Le Tri Trung[1][0009-0007-0790-1388], Ha Van An[2][0009-0000-8536-4396], and Nguyen Thi Thuy Hoai[3][0009-0007-6328-0715]

[1] FPT University, Da Nang, Vietnam
letritrung2605@gmail.com
[2] FPT University, Da Nang, Vietnam
antruongkiet1412@gmail.com
[3] FPT University, Da Nang, Vietnam
hoaintt40@fe.edu.vn

Abstract. Concurrency bugs in asynchronous JavaScript (Node.js) code are a leading cause of serious failures in production systems, including data races, lost updates, and application hangs. Existing tools fall into two categories: rule-based static detectors that can identify suspicious patterns but cannot generate fixes, and large language models (LLMs) that can reason about code but tend to miss the exact buggy line and require large model sizes (7B–32B parameters) to perform well. ThreadLearn addresses both weaknesses by combining a 5-pattern static race detector with a small language model (Qwen2.5-Coder-1.5B [6]) fine-tuned using QLoRA on 892 training examples with hindsight Chain-of-Thought reasoning. At inference time, a BM25 retrieval pipeline fetches the 3 most relevant documents from a 2,050-document knowledge base to augment the prompt. On a 30-case real-world benchmark drawn entirely from production npm packages (rw_01–rw_30) spanning 10 concurrency bug categories, ThreadLearn with the full BM25+AST pipeline achieves **22.0/30 (73.3%)**, compared with 60.0% for the base model without fine-tuning and 65.0% for GPT-3.5-turbo with the same pipeline—despite having $\sim$125$\times$ fewer parameters. A key finding is that the retrieval pipeline only benefits models that already have domain knowledge: the base model gains no benefit from retrieval while the fine-tuned model gains +10 percentage points. The fine-tuning dataset and evaluation scripts are publicly available.

Keywords: concurrency bugs · race conditions · static analysis · LLM fine-tuning · retrieval-augmented generation · JavaScript

1 Introduction

Asynchronous JavaScript (Node.js [13]) is widely used for building web backends, APIs, and real-time services, but the event-driven, non-blocking execution model

makes it easy to write code with subtle concurrency bugs. A large-scale empirical study (NodeCB [1]) analyzed 57 real concurrency bugs across 53 open-source Node.js projects and found that 65% involve atomicity violations, 30% involve order violations, and 93% cause serious consequences such as crashes, corrupted database state, or process hangs.

Despite how common and harmful these bugs are, existing automated tools still fall short. Rule-based static analyzers (e.g., ESLint async rules, ThreadSanitizer) can detect suspicious patterns but cannot explain or fix them, and they generate false alarms on modern `async`/`await` code. LLM-based approaches [2] can reason about code semantics but cannot pinpoint the exact buggy line without structural guidance, and they require large models to achieve competitive accuracy.

We identify four concrete gaps in the state of the art:

G1 Rule-based detectors find bugs but cannot generate fixes.
G2 LLM-only approaches ignore structural signals from static analysis and require 7B–32B parameter models.
G3 No existing tool uses static detector output as structured context to guide LLM reasoning.
G4 No end-to-end tool covers the full workflow of *detect* → *explain* → *fix* for JavaScript (Node.js).

This paper makes four contributions:

1. `race_detector`: a 5-pattern static analyzer for JavaScript (Node.js) with an AST-based code preprocessor (§4.3).
2. **Hindsight CoT dataset**: 892 training examples of the form `(code, detector_output, reasoning_trace, fix)`, where reasoning is written after the correct fix is known (§4.3).
3. **ThreadLearn pipeline**: an end-to-end system that detects, retrieves context, and generates a fix, served via a FastAPI web endpoint (§4.3).
4. **Evaluation**: comparison against a base LLM and GPT-3.5-turbo on a 30-case real-world JavaScript concurrency benchmark across fix pass rate, per-category accuracy, and ablation study (§5).

Guided by the gaps above, we structure our evaluation around three research questions:

RQ1 Does combining a static race detector with a fine-tuned small language model outperform a general-purpose LLM (GPT-3.5-turbo) on real-world JavaScript concurrency bugs, despite having far fewer parameters? (answered in §5 and §5)
RQ2 Does BM25+AST retrieval augmentation benefit every model equally, or does its benefit depend on whether the model has been fine-tuned on domain-specific data? (answered in §5–§5)
RQ3 Which concurrency bug categories remain hard for a fine-tuned small model, and what does that reveal about the limits of pattern-based training data? (answered in §5.1)

Table 1: Concurrency bug types in Node.js (from NodeCB [1]).

Bug Type	Rate	Manifestation in Node.js
Atomicity violation	65%	Two async operations overlap on shared state (e.g., lost database update)
Order violation	30%	Callback fires before dependent operation completes
Starvation	5%	I/O queue exhausted; deferred tasks never execute
Closure loop var	JS	`var i` shared by reference across all `setTimeout` callbacks

The remainder of the paper is organized as follows. §2 introduces the Node.js concurrency bug taxonomy, BM25 retrieval, and QLoRA fine-tuning that underpin our design. §3 positions ThreadLearn against rule-based and LLM-based prior work. §4 describes the dataset, the static detector, the hindsight Chain-of-Thought fine-tuning procedure, and the end-to-end pipeline architecture. §5 reports experimental results, directly answering RQ1–RQ3. §6 summarizes findings and outlines future work.

2 Background

Concurrency bug taxonomy. Concurrency bugs in asynchronous JavaScript arise from the single-threaded event loop model: a single thread interleaves callback execution, I/O completion, and timer firing in an order that depends on runtime scheduling rather than the order code is written. Unlike race conditions in multi-threaded languages, which stem from unsynchronized memory access across CPU cores, Node.js concurrency bugs stem from *interleaving* async operations whose relative completion order is not guaranteed—two database writes issued back-to-back may resolve in either order depending on network latency, queue depth, or event-loop phase. Table 1 summarizes the four recurring bug types identified in the NodeCB study [1] and how each manifests concretely in Node.js code.

Atomicity violations dominate (65%) because Node.js encourages fire-and-forget I/O: two logically atomic steps (e.g. check-then-update) execute as separate, interruptible operations. Order violations (30%) occur when a callback assumes a prior async step completed (e.g. reading a file before a preceding write flushed), and starvation (5%) is rarer but harder to diagnose since the hung-request symptom appears far from its saturated-queue cause. The fourth row, *closure loop var*, is a JavaScript-specific pitfall rather than a NodeCB category: function-scoped `var` makes a loop variable referenced inside a deferred callback observe its *final* value rather than the value at scheduling time. We include it because, though syntactic, it is among the most frequent bugs in our benchmark (§4.1) and is fully fixable by a deterministic rewrite (`var`→`let`)—an ideal first target for rule-based detection.

Table 2: Comparison of ThreadLearn with related tools.

Tool	Detect	Fix	JS	Fine-tuned
ThreadSanitizer	✓	×	×	×
ESLint async	✓	×	✓	×
NodeCB (rules)	✓	×	✓	×
PCWMs (LLM-only)	✓	✓	✓	✓
ThreadLearn	✓	✓	✓	✓

BM25 retrieval. BM25 [7] scores a document D against query $Q = \{q_1, \ldots, q_n\}$ by term frequency and inverse document frequency with length normalization:

$$\text{BM25}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D| / \overline{|D|})}$$

with $k_1 = 1.5$, $b = 0.75$. We use BM25 (via `rank-bm25` [18]) over dense vector search because exact API name matching (e.g. `setTimeout`) is more precise than semantic similarity for concurrency patterns.

QLoRA fine-tuning. QLoRA [3] fine-tunes a 4-bit NF4-quantized model via LoRA [9] adapters, updating only the adapter weights so training fits on a single laptop GPU. We use rank $r = 16$, scaling $\alpha = 32$.

3 Related Work

Rule-based detectors match code against predefined patterns. ThreadSanitizer [4] detects data races at runtime but only on C/C++ and Java; ESLint [12] adds async rules for JavaScript but only syntactic ones, missing semantic races across callbacks; the NodeCB regex matcher [1] has high false-positive rates on modern `async`/`await` code; and dynamic delay-injection (NACD [5]) exposes more event races but still generates no fixes. *ML/LLM approaches* train on code features for defect detection, but pre-trained code models are not concurrency-specific; PCWMs [2] applies reasoning world models to parallel code yet relies on 7B–32B models and ignores static structure; and RAG code tools [8] improve completion but have not been applied to JavaScript concurrency remediation. Table 2 positions ThreadLearn against these tools along four axes: detection, fix generation, JavaScript support, and fine-tuning.

The rule-based tools flag a line but leave the fix to the developer; PCWMs generates fixes but without structural signal, so it must infer both *where* and *how* from semantics alone (hence 7B–32B parameters). ThreadLearn is the only entry combining all four axes: its detector narrows the search to a specific `pattern_id` and `line_range` before the LLM runs, letting a 1.5B model match a far larger general-purpose model (§5).

Table 3: Real-world benchmark distribution (30 cases from production npm packages).

Category	Count	Example Package
Race Condition	5	`request`, `passport`, `bull`
Double Callback	4	`mysql`, `ioredis`, `async`
Unhandled Rejection	5	`express`, `mongoose`, `co`
Resource Exhaustion	4	`pg`, `axios`, `sequelize`
Event Loop Blocking	3	`express` (sync middleware), Node.js crypto
Sequential Awaits	3	`express-session`, nodebestpractices [16]
Zalgo	2	`async`
Context Loss	2	`express-async-errors`, Node.js timers
Stream Leak	1	Node.js streams
Callback Hell	1	`async`
Total	**30**	

4 Proposed Approach

4.1 Dataset

Our study uses three datasets. The **fine-tuning dataset** (892 examples) has the form `(code, detector_output, reasoning_trace, fix)`, where each reasoning trace is written by a teacher model *after* the correct fix is known (hindsight CoT), and was built entirely by the authors with no crowdsourcing or model-generated labels. It comprises 332 synthetic pairs (37.2%)—handcrafted and manually verified—and 560 generated pairs (62.8%) produced by `generate_pairs()`, a template engine that expands the handcrafted pairs over 40 domain slots (e.g. `User`, `Order`) using 18 generator functions, each a distinct concurrency transformation (e.g. sequential `await`→`Promise.all`, callback→`async`, unguarded `fetch`→`AbortController` timeout). Because each generator is a deterministic template, all generated labels are correct by construction, avoiding the label noise of LLM-sampled data while teaching the *pattern* rather than specific names.

Knowledge base. The retrieval knowledge base contains 2,050 JavaScript documents collected from authoritative sources: MDN Web Docs [15] (99 documents covering `Promise`, `async`/`await`, and Node.js API behavior), official documentation of concurrency libraries (RxJS, p-limit, async-mutex, Bluebird; 50 documents), curated ThreadLearn pattern descriptions (151 documents), and 1,750 synthetic documents generated via context permutation (re-phrasing API descriptions with synonyms and varied identifiers) to broaden BM25 coverage of API name variants. All 2,050 documents fall into three groups: `patterns` (1,418), `race-conditions` (352), and `anti-patterns` (280).

Real-world evaluation benchmark. To avoid bias from self-generated test cases, we built a benchmark of 30 real-world JavaScript concurrency bugs drawn entirely from production open-source packages. Table 3 summarizes the distribution across 10 bug categories.

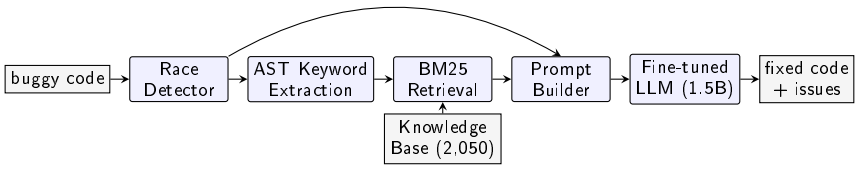

Fig. 1: ThreadLearn end-to-end pipeline. Buggy JavaScript code flows through five modules: static race detection, AST keyword extraction, BM25 retrieval from a 2,050-document knowledge base, prompt construction, and fine-tuned LLM inference to produce both a diagnosis and a corrected fix.

Every case (rw_01–rw_30) traces to a specific GitHub issue or Node.js API document (e.g. `request#2484`, `mysqljs#1260`), so no case is synthetic; drawing them from independent production incidents rather than the training-data engine eliminates model-favorable bias. The full case-to-source list is in the public repository.

4.2 Data Preprocessing and Evaluation Strategy

AST Preprocessing. ThreadLearn's preprocessor provides four esprima-based [11] helpers shared by the detector and retrieval stage: `stripComments` and `normalizeWhitespace` canonicalize the source so the detector's regex patterns and BM25 query are not thrown off by formatting; `extractFunctions` scopes detector line-ranges to a function body; and `extract_keywords` returns the top-20 identifier names as the literal BM25 query—so preprocessing quality directly affects retrieval precision.

Evaluation strategy. Each test case specifies an expected fix pattern (e.g. `Promise.all`, `try/catch`, `let` in loop) and is scored **PASS** if the generated code contains it, **PARTIAL** if code is generated without the pattern, and **FAIL** if no code is generated.

4.3 Model Architecture

ThreadLearn has five independent modules that communicate through simple data structures, allowing any module to be replaced independently. Figure 1 shows the end-to-end pipeline, which follows five sequential steps:

The five steps are: (1) **static detection**—`race_detector` scans the source in $O(n)$ time, emitting `{pattern_id, line_range, description}` per match (§4.3); (2) **keyword extraction**—`extract_keywords()` walks the esprima AST and collects `Identifier` tokens as the BM25 query, keeping CamelCase API names (e.g. `setTimeout`) intact rather than splitting them; (3) **retrieval**—BM25 returns the top-3 of 2,050 knowledge-base documents; (4) **prompt construction**—a `prompt_builder` assembles the detector report, retrieved documents, and buggy code into a structured prompt; (5) **fix generation**—the fine-tuned LLM outputs corrected code with step-by-step reasoning.

Static Race Detector. The detector checks 5 JavaScript concurrency patterns in two groups. *Closure/async*: `closure_loop_var` (`var` captured by reference in loop callbacks), `shared_var_settimeout` (shared variable modified inside `setTimeout`), and `promise_no_await` (Promise created but not awaited). *Shared state*: `concurrent_write_array` (array mutated from concurrent callbacks) and `counter_no_atomic` (non-atomic counter increment). For example, the `closure_loop_var` pattern fires when a `var` loop variable is captured inside a `setTimeout` or Promise callback, so all callbacks read the final loop value; the fix replaces `var` with `let` to create a per-iteration binding.

Listing 1.1: `closure_loop_var`: bug (`var`, prints 5,5,...) vs. fix (`let`, prints 0,1,...).

```
for (var i = 0; i < 5; i++)              // bug: always prints 5
  setTimeout(() => console.log(i), 100);
for (let i = 0; i < 5; i++)              // fix: prints 0,1,2,3,4
  setTimeout(() => console.log(i), 100);
```

Hindsight Chain-of-Thought Fine-Tuning. Training examples have the form:

$$(\textit{code},\ \textit{detector\_output},\ \textit{reasoning\_trace},\ \textit{fix})$$

The *reasoning_trace* follows the Chain-of-Thought paradigm [17] but is written by a teacher model *after* the correct fix is known (hindsight reasoning), which yields a cleaner training signal than forward reasoning under uncertainty. We fine-tune Qwen2.5-Coder-1.5B [6] with QLoRA (r=16, α=32, 4-bit NF4) via `SFTTrainer` [10] on the 892 examples, grounding each on the detector's `pattern_id` and `line_range`, on a single NVIDIA RTX 4060 laptop GPU. This grounding—rather than raw code alone—is the key design choice: it lets a 1.5B model learn a tractable mapping from a small, discrete set of bug patterns to fixes instead of general concurrency reasoning from scratch.

5 Experimental Results and Discussions

All experiments use the 30-case benchmark (§4.1), run on a Kaggle T4 GPU for local models and the OpenAI API for GPT-3.5-turbo [14]. We evaluate six configurations—{base, fine-tuned, GPT-3.5-turbo} × {no-pipeline, +pipeline}, where the pipeline is AST keyword extraction → BM25 top-3 retrieval → augmented prompt—scored PASS=1.0, PARTIAL=0.5, FAIL=0. This staged design isolates the value of fine-tuning (models without pipeline) from that of retrieval (each model with vs. without pipeline).

Table 4 reports all six configurations. Without any pipeline, the three models score within a narrow band (60.0–63.3%) with zero FAIL cases: every model emits syntactically valid code, and the dominant failure mode is surface-level

Table 4: Full ablation on 30-case real-world benchmark (score = PASS + 0.5×PARTIAL).

Configuration	Pass	Partial	Fail	Score/30	%
Base (no fine-tune, no pipeline)	6	24	0	18.0	60.0%
GPT-3.5-turbo (no pipeline)	7	23	0	18.5	61.7%
Fine-tuned only (no pipeline)	8	22	0	19.0	63.3%
Base + pipeline	8	20	2	18.0	60.0%
GPT-3.5-turbo + pipeline	9	21	0	19.5	65.0%
ThreadLearn + pipeline	**14**	**16**	**0**	**22.0**	**73.3%**
Fine-tuning contribution (no pipeline):				+1.0	+3.3 pp
Pipeline contribution (fine-tuned model):				+3.0	+10.0 pp
Combined gain (ThreadLearn vs. base):				+4.0	+13.3 pp

rewriting into `async`/`await` syntax without addressing the underlying hazard. ThreadLearn merged already edges out GPT-3.5-turbo here (+0.5 pt), confirming **G2**—without structural guidance even a strong general-purpose LLM cannot reliably pinpoint the correct fix. Adding the BM25+AST pipeline then separates the models sharply: the fine-tuned model gains +3.0 pts (+10 pp), GPT-3.5-turbo gains only +1.0 pt, and the base model gains *nothing* (60.0% in both conditions) while introducing 2 FAIL cases.

Three conclusions follow. First, **fine-tuning is a prerequisite for pipeline benefit**: the base model cannot use retrieved documentation constructively (0 pp, 2 new FAILs). Second, **the pipeline amplifies fine-tuning**—it adds +3.0 pts on top of fine-tuning's +1.0 pt, a 3× larger gain. Third, **ThreadLearn (73.3%) beats GPT-3.5-turbo+pipeline (65.0%) by +2.5 pts despite ∼125× fewer parameters** (1.5B vs. ∼175B).

Answering RQ1 and RQ2. The third conclusion answers **RQ1**: combining the static detector with a fine-tuned 1.5B model outperforms GPT-3.5-turbo despite the parameter disadvantage, because the detector's structured output narrows what the LLM must infer. The first answers **RQ2**: BM25+AST retrieval does *not* benefit every model equally—+10 pp for the fine-tuned model versus 0 pp (and regression) for the base model—so retrieval augmentation only pays off once a model already has the domain knowledge to exploit it.

5.1 Per-Category Analysis and RQ3

Breaking the 22.0/30 score down by category, ThreadLearn is strongest on Unhandled Rejection (90%), Sequential Awaits (100%), and Race Condition (70%, up from 50% for both baselines)—categories well-represented in training and tied to specific fix templates (`try`/`catch`, `Promise.all`, `var`→`let`). It is weakest on Zalgo, Double Callback, and Stream Leak (all 50%), the last being the only category where it trails the base model. This answers **RQ3**: these categories remain hard because their fixes require tracing *callback invocation order* across asynchronous boundaries—a semantic property not inferable from local

syntactic context—whereas our template-generated training data (§4.1) is dominated by syntactic substitutions. Pattern-template data thus generalizes well to syntactically-fixable bugs but underrepresents bugs needing multi-step temporal reasoning.

On latency, ThreadLearn (≈26 s/case) is ∼14× slower than the GPT-3.5-turbo API (≈1.8 s) with <0.5 s pipeline overhead—a cost–privacy trade-off, since local inference runs on-device at no per-call cost with no data leaving the machine.

6 Conclusion and Research Directions

ThreadLearn addresses the gap between static detection and automated remediation by combining a 5-pattern rule-based race detector with a small language model (Qwen2.5-Coder-1.5B) fine-tuned on 892 hindsight Chain-of-Thought examples and a BM25+AST retrieval pipeline. On a 30-case real-world benchmark drawn entirely from production npm packages, ThreadLearn achieves **22.0/30 (73.3%)**—a +13.3 pp gain over the base model and +8.3 pp over GPT-3.5-turbo with the same pipeline, despite ∼125× fewer parameters. It is the first end-to-end system to pair a rule-based race detector with a fine-tuned small language model and RAG for JavaScript concurrency bug remediation, bridging PCWMs [2] (LLM-only, large model) and rule-based detectors (ESLint, ThreadSanitizer) that produce no fixes. The ablation establishes a design principle: *retrieval only helps models that already have domain knowledge*, so pipeline augmentation should be paired with domain-specific fine-tuning rather than applied standalone.

Future work will (1) expand the static detector to cover TypeScript AST patterns; (2) add mutex and semaphore patterns for broader concurrency coverage; and (3) investigate whether a larger fine-tuned model (e.g., Qwen2.5-Coder-7B) would push Unhandled Rejection and Race Condition above 75%.

References

1. J. Wang, W. Dou, Y. Gao, C. Gao, F. Qin, K. Yin, and J. Wei. A Comprehensive Study on Real World Concurrency Bugs in Node.js (NodeCB). In *Proc. 32nd IEEE/ACM ASE*, pp. 520–531, 2017.
2. G. Singh, A. Guha, B. Kailkhura, and H. Menon. Learning Reasoning World Models for Parallel Code. *arXiv:2604.20926*, 2026.
3. T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *Proc. NeurIPS*, 2023.
4. K. Serebryany and T. Iskhodzhanov. ThreadSanitizer: Data Race Detection in Practice. In *Proc. WBIA*, pp. 62–71, 2009.
5. A. T. Endo and A. Møller. Event Race Detection for Node.js Using Delay Injections. In *39th European Conference on Object-Oriented Programming (ECOOP 2025)*, pp. 9:1–9:28, 2025.
6. Qwen Team. Qwen2.5-Coder Technical Report. *arXiv:2409.12186*, 2024.

7. S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
8. P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proc. NeurIPS*, 2020.
9. E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. In *Proc. ICLR*, 2022.
10. L. von Werra, Y. Belkada, L. Tunstall, et al. TRL: Transformer Reinforcement Learning. `https://github.com/huggingface/trl`, 2020.
11. A. Hidayat. Esprima: ECMAScript Parsing Infrastructure. `https://esprima.org`, 2012.
12. N. C. Zakas. ESLint: The Pluggable JavaScript Linter. `https://eslint.org`, 2013.
13. R. Dahl. Node.js: Evented I/O for V8 JavaScript. `https://nodejs.org`, 2009.
14. OpenAI. GPT-3.5 Turbo. `https://platform.openai.com/docs/models/gpt-3-5-turbo`, 2023.
15. Mozilla. MDN Web Docs: JavaScript Reference. `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, 2024.
16. Y. Goldberg et al. The Node.js Best Practices Repository. `https://github.com/goldbergyoni/nodebestpractices`, 2017.
17. J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Proc. NeurIPS*, 2022.
18. D. Brown. rank-bm25: A Collection of BM25 Algorithms in Python. `https://github.com/dorianbrown/rank_bm25`, 2020.